Fused Block-Sparse Attention with Learned Content-Dependent Selection:

A Triton Kernel for Memory-Efficient Long-Context Inference

Miroslav Drbal
mdrbal@nymfe.net
Gen Digital, Inc. | miroslav.drbal@gendigital.com

Abstract

We present a block-sparse attention mechanism with a fused Triton kernel that achieves memory parity with dense FlashAttention while delivering up to 7.0× latency speedup at 262k-token context length. The block selector is trained jointly with the model via MoE-style gate coupling and an auxiliary routing loss, reaching near-perfect block-retrieval accuracy (hit = 0.99) without a separate pre-training stage. On a 45M-parameter language model trained on TinyStories, the sparse model achieves validation perplexity 15.0 versus 14.4 for the dense baseline, a 4% gap that narrows to parity at shorter context windows. We also investigate centroid-based linear selection ($\mathcal{O}(N \cdot C)$) as a replacement for the $\mathcal{O}(N^2/B_s^2)$ block-pair selection, finding that InfoNCE contrastive training fixes a critical collapse failure in the routing loss (hit $0.19 \rightarrow 1.0$) but that task accuracy remains bottlenecked by selection precision, not routing recall. We further isolate selection from the attention read up to 16M tokens and show that single-level block-pair selection has an $\mathcal{O}(N^{4/3})$ compute floor: no block-size schedule yields end-to-end linearity, as the quadratic only moves between the selection and read stages. A second, lossless linear selector, LSH bucketing, matches block-sparse accuracy under supervised routing (0.92 vs. 0.96 on MQAR) and far exceeds centroid. Training through a dense selector and hashing only at inference is linear and accuracy-preserving for retrieval; for language modeling a parameter-free hash collapses at a sparse budget (perplexity 349), but a learned, co-trained hash restores usability to within 30% of the quadratic selector (perplexity 10.2), recovering at block granularity the token-level advantage of learned over parameter-free hashing. Finally, we validate the mechanism on a frontier model: swapping our training-free block-sparse selection into the 10 full-attention layers of Qwen3.6-35B-A3B (a 35B parameter mixture-of-experts model) recovers full-attention perplexity to within 1.6% at 64k context while attending to only 6% of blocks, and beats random selection by 3 to 4× at matched budget; single-needle retrieval needs roughly twice that block budget under mean-query selection, a gap a one-line last-token selector largely closes. All experiments run on a single NVIDIA GB10 Grace-Blackwell GPU.

1 Introduction

Transformer-based language models face a quadratic attention bottleneck: standard softmax attention scales as $\mathcal{O}(N^2)$ in sequence length N, limiting practical context windows. While FlashAttention [1, 2] reduces the *constant* factor by tiling computations in SRAM, the asymptotic cost remains quadratic.

Content-dependent sparse attention offers a path to genuine sub-quadratic scaling by selecting a small subset of key/value blocks per query block. Mechanisms such as Native Sparse Attention (NSA) [3], MoBA [4], and the Routing Transformer [5] learn to route queries to relevant key blocks, reducing the attention cost to $\mathcal{O}(N \cdot k \cdot B_s)$ where k is the number of selected blocks and B_s is the block size.

However, two practical barriers remain:

1. **Selector training.** Discrete top-k selection severs the gradient path to the selector parameters. Without auxiliary supervision, the selector remains a frozen random projection.
2. **Gather overhead.** Even with correct selection, the reference PyTorch implementation materializes gathered key/value tensors, consuming up to 40× more memory than dense FlashAttention and negating the theoretical savings.

We address both barriers. For (1), we use MoE-style gate coupling [6], adding log-sigmoid(score) as a bias to attention logits, so the selector receives task gradients. An auxiliary routing loss (cross-entropy on the known needle block for synthetic tasks, InfoNCE contrastive loss [14] for centroid routing) provides cold-start supervision. For (2), we implement a fused Triton kernel [13] that reads selected block indices, gathers key/value blocks one at a time from the KV cache, and computes online-softmax in SRAM, never materializing the gathered tensor.

Contributions.

- A **fused Triton kernel** for block-sparse attention (forward + backward) achieving memory parity with dense FlashAttention (1.11 GB vs. 1.12 GB at 262k tokens) and 7.0× latency speedup (Section 4).
- **Gate-coupled selector training** with InfoNCE routing loss, fixing a collapse failure in centroid-based linear selection (hit 0.19 → 1.0) and characterizing the precision/recall tradeoff (Section 5).
- An **end-to-end language model** (45M params, TinyStories [12]) trained with the fused kernel, achieving val perplexity 15.0 vs. 14.4 dense (Section 6).
- A **negative result**: linear-selection centroid routing achieves perfect retrieval recall but cannot match block-sparse task accuracy, identifying representation quality under coarse routing as the bottleneck (Section 5).
- A **scaling analysis** isolating selection from the attention read to 16M tokens: the read is linear, but single-level block-pair selection has an $\mathcal{O}(N^{4/3})$ floor that no block-size schedule can beat and that exhausts memory before any latency crossover (Section 7.1).
- A **learned-hash linear selector**: training through a dense selector and hashing only at inference is linear and accuracy-preserving for retrieval, and a co-trained hash recovers language-model usability that a parameter-free hash loses at a sparse budget (perplexity 349 → 10.2), at a tunable representation penalty (Section 5.6).
- A **frontier-model validation**: swapping the training-free selector into the 10 full-attention layers of Qwen3.6-35B-A3B (35B-parameter MoE) holds within 1.6% of full-attention perplexity at 64k while attending to 6% of blocks and beats random selection 3 to 4× at matched budget. An oracle ceiling bounds the achievable gain (a perfect selector roughly halves the heuristic's loss), and a distilled linear selector captures none of it; single-needle retrieval needs about twice the perplexity budget under mean-query selection, a gap a one-line last-token query reduction largely closes (Section 8).

2 Background and Related Work

Sparse attention. Fixed-pattern sparsity (Longformer [7], BigBird [8]) restricts attention to local windows plus global tokens but cannot solve content-dependent retrieval tasks like multi-query associative recall (MQAR) [15, 11]. Content-dependent methods such as the Routing Transformer [5], Reformer [9], Sparse Sinkhorn Attention [10], NSA [3], and MoBA [4] learn to select relevant blocks but face the selector-gradient problem.

Inference-time sparse selection. A parallel line keeps a dense-trained model and sparsifies only at inference, selecting a query-dependent subset of the KV cache: Quest [16] (top-k pages), H2O [17] (heavy-hitter eviction), and RetrievalAttention [18] (attention-aware ANN over keys, which reports that plain LSH/ANN underperforms because attention queries are out-of-distribution). LSH-E [19] uses SimHash for KV eviction, and DASH-KV [20] replaces LSH with a *learned* asymmetric hash, reporting higher retrieval recall than parameter-free LSH. Our train-dense / infer-LSH study (Section 5.6) sits in this regime, at block rather than token granularity and at small scale.

Selector training. NSA trains the selector by distilling dense per-block attention mass. MoBA uses a gating mechanism. Sparse Sinkhorn Attention [10] uses differentiable sorting via the Sinkhorn operator to learn latent permutations. We build on the MoE gate-coupling approach [6]: the selector score is injected as a log-sigmoid bias into the attention logits, creating a differentiable path. An auxiliary routing loss provides cold-start supervision.

Flash-style fusion. FlashAttention [1] tiles Q/K/V into blocks and computes online-softmax in SRAM, achieving exact attention with reduced memory I/O. FlashAttention-2 [2] improves parallelism and work partitioning. Our Triton kernel applies the same dataflow but iterates over *selected* key blocks via an index list rather than all blocks in sequence order.

Centroid routing. Routing Transformer [5] assigns tokens to k-means clusters. Reformer [9] uses locality-sensitive hashing. Our CentroidSSA routes blocks through learned centroids ($\mathcal{O}(N \cdot C)$) but uses InfoNCE contrastive training [14] instead of k-means, addressing the non-differentiability of hard cluster assignment.

Triton. Triton [13] is an intermediate language and compiler for tiled neural network computations, enabling custom GPU kernels at near-CUDA performance from Python. Our forward and backward kernels are implemented in Triton 3.7.

3 Method

3.1 Block-Sparse Attention with Gate Coupling

Given input $x \in \mathbb{R}^{B\times L\times D}$, we partition the sequence into $n_{\text{blk}} = L/B_s$ blocks of size B_s. Per layer:

1. **Projections.** $Q, K, V = \text{QKV}(x)$, reshaped to (B, H, L, d_h).
2. **Block scores.** A small MLP selector computes block-level summaries:

$$s_q = \text{mean}(\text{sel\_q}(x)_{\text{blk}}) \in \mathbb{R}^{B\times n_{\text{blk}}\times d_s} \tag{1}$$

$$s_k = \max\big(\text{mean}(\text{sel\_k}(x)_{\text{blk}}),\ \max(\text{sel\_k}(x)_{\text{blk}})\big) \tag{2}$$

$$S = \max(s_q s_{k,\text{mean}}^{\top},\ s_q s_{k,\max}^{\top}) \in \mathbb{R}^{B\times n_{\text{blk}}\times n_{\text{blk}}} \tag{3}$$

3. **Selection.** For each query block i, select the top-k key blocks by $S[i,\cdot]$, plus the own (diagonal) block. Causal masking excludes future blocks. Result: $\text{sel} \in \mathbb{Z}^{B\times n_{\text{blk}}\times k_{\text{tot}}}$.
4. **Gate coupling.** Add log-sigmoid$(S[i,j])$ as a bias to the per-token attention logits for selected block j:

$$\text{logit}_{i,j} = \frac{Q_i K_j^{\top}}{\sqrt{d_h}} + \text{log-sigmoid}(S_{i,j}) \tag{4}$$

   This makes the output depend differentiably on S, so sel_q and sel_k receive task gradients.
5. **Attend.** Exact scaled-dot-product attention over the selected blocks only ($k_{\text{tot}} \cdot B_s$ keys per query block).

3.2 Auxiliary Routing Loss

For synthetic tasks (MQAR) where needle positions are known, we supervise the selector with cross-entropy to the true needle block:

$$\mathcal{L}_{\text{route}} = \text{CE}\big(\text{softmax}(S_{\text{last}}),\ \lfloor \text{needle\_pos}/B_s \rfloor\big) \tag{5}$$

For centroid routing (Section 5), we replace this with an InfoNCE contrastive loss. The total loss is $\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_r \mathcal{L}_{\text{route}}$, with λ_r annealed to zero over the second half of training.

3.3 Complexity

- **Attention:** $\mathcal{O}(N \cdot k \cdot B_s)$, linear in N.
- **Selection (block-pair):** $\mathcal{O}(n_{\text{blk}}^2) = \mathcal{O}((N/B_s)^2)$, quadratic on the reduced axis, shrunk by B_s^2.
- **Selection (centroid):** $\mathcal{O}(n_{\text{blk}} \cdot C)$, linear.
- **Memory (fused kernel):** $\mathcal{O}(N)$, only base Q/K/V, no gathered tensor.

4 Fused Triton Kernel

4.1 Forward Kernel

Each Triton program handles one (B, H, q_{blk}) tile. It loads the $B_s \times d_h$ query tile, then iterates over the k_{tot} selected key blocks via the index list sel. For each selected block:

1. Load K, V tiles $(B_s \times d_h)$ from the KV cache at the indexed block offset.
2. Compute $B_s \times B_s$ attention scores with scale.
3. Apply causal mask (triu) for the own block; mask future blocks entirely.
4. Add gate bias log-sigmoid$(S_{i,j})$ if enabled.
5. Update online-softmax state (running max m, running sum ℓ, accumulator acc) via the standard FlashAttention recurrence [1].

The output is acc/ℓ. No gathered K/V tensor or score matrix is ever materialized, all intermediate state lives in SRAM.

4.2 Backward Kernel

The backward pass follows the FlashAttention-2 recompute pattern [2] with three iterations over the selected blocks:

1. **Pass 1:** Recompute forward to obtain final m and ℓ.
2. **Pass 2:** Compute $D = \sum_j \text{rowsum}(P_j \cdot dP_j)$, the total diagonal term for the softmax Jacobian across all selected blocks.
3. **Pass 3:** Compute gradients:

$$dV = P^\top dO \tag{6}$$
$$dP = dO\, V^\top \tag{7}$$
$$dS = P \odot (dP - D) \cdot \text{scale} \tag{8}$$
$$dQ \mathrel{+}= dS\, K, \quad dK \mathrel{+}= dS^\top Q \tag{9}$$

dK and dV are written via `atomic_add` (multiple query blocks may select the same key block); dQ is written directly.

4.3 Integration

A custom `torch.autograd.Function` (`BSAttnFunction`) routes forward and backward through the Triton kernels. The `BlockSparseSSA(use_triton=True)` flag switches between the PyTorch reference and the fused path. Non-contiguous tensors from the QKV projection are made contiguous before the kernel call.

4.4 Verification

- **Forward:** 9.77×10^{-4} max diff vs. PyTorch SDPA (fp16, B=2, H=4, L=64, d_h=32, k=3, causal).
- **Backward:** dQ: 4.0×10^{-3}, dK: 4.4×10^{-3}, dV: 3.7×10^{-3} vs. PyTorch autograd (fp32, k=3, causal).
- **Integration:** forward 4.5×10^{-4}, gradient 1.1×10^{-3} vs. PyTorch path (same model, toggle `use_triton`).
- **Gate bias:** 9.77×10^{-4} with gate enabled.

5 Linear Selection: Centroid and LSH

Block-pair selection scales as $\mathcal{O}(n_{\text{blk}}^2)$. To remove this quadratic term, we propose CentroidSSA: route blocks through C learned centroids, analogous to k-means routing in the Routing Transformer [5] but with differentiable training.

5.1 Mechanism

Each block is assigned to its nearest centroid (argmax of $s_k \cdot \text{cent}^\top$). Each query block routes to its top-c_{pq} centroids and attends to all blocks in those buckets (capacity c_{cap} per bucket). Selection cost: $\mathcal{O}(n_{\text{blk}} \cdot C)$, linear.

5.2 The Circular Loss Failure

The original routing-alignment loss was:

$$\text{consensus} = \text{argmax}(s_q + s_k), \quad \mathcal{L} = \text{CE}(s_q, \text{consensus}) + \text{CE}(s_k, \text{consensus}) \tag{10}$$

This is *circular*: at cold start, s_q and s_k are near-uniform, so argmax is arbitrary noise. The loss collapsed to $\ln C$ (uniform), and retrieval hit stayed at 0.19.

5.3 InfoNCE Fix

We replace the circular loss with an InfoNCE contrastive objective [14]:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(q_{\text{last}}, k_{\text{needle}})/\tau)}{\sum_j \exp(\text{sim}(q_{\text{last}}, k_j)/\tau)} \tag{11}$$

where $\text{sim}(q, k) = \text{softmax}(q/\tau) \cdot \text{softmax}(k/\tau)^\top$ and $\tau = 0.5$. The positive is the needle key block; all other key blocks are negatives. This is fully differentiable and has no argmax dependency.

5.4 Capacity Sweep

Table 1: Centroid capacity sweep (8-pair MQAR, 10k steps, curriculum). C = centroids, cap = bucket capacity.

C	cap	hit	acc
16	4	0.71	0.16
16	8	**1.00**	0.15
32	4	0.68	0.16
32	8	**1.00**	0.16
64	4	0.68	0.17
64	8	**1.00**	0.17
Block-sparse (baseline)		0.99	**0.77**

Table 1 shows that **capacity is the dominant variable**: cap = 4 → hit ≈ 0.70; cap = 8 → hit = 1.0. Centroid count (16 → 64) has negligible effect. InfoNCE + cap=8 achieves selector parity with block-sparse (hit= 1.0), but task accuracy is stuck at ∼0.16 vs. block-sparse's 0.77.

5.5 The Precision/Recall Bottleneck

With cap=8, each query block selects $c_{pq} \times \text{cap} + 1 = 17$ blocks. The needle is always in the set (hit=1.0) but surrounded by 12 extra distractor blocks. We tested three remedies:

- **Within-bucket refinement** (top-k from candidates): hit drops to 0.72, acc unchanged (0.17).
- **Train-high / inference-low** (train cap=8, eval cap=4): hit drops to 0.73, acc drops to 0.15.
- **Main q/k refinement** (score candidates with attention q/k instead of selector): hit 0.72, acc 0.17.

None closed the gap. The bottleneck is *not* routing precision but *representation quality*: the coarse centroid routing path shapes the attention's q/k/v projections differently than block-sparse's direct pairwise scoring, limiting token-level extraction even when the needle block is present.

5.6 LSH Bucketing: Train Dense, Infer Linear

Centroid routing loses information in the *representation* (queries are compared against lossy cluster summaries). A second linear-time selector, LSH bucketing [9], instead prunes the *search*: block representations are hashed by fixed random projections into buckets (n_{rounds} rounds), a query block reads its own bucket, and the candidates are re-scored with the full $s_q \cdot s_k$ cosine and truncated to block-sparse's budget, so a selected block is scored at full fidelity. Selection is $\mathcal{O}(n_{\text{blk}} \cdot n_{\text{rounds}} \cdot (n_{\text{buckets}}+\text{cap}))$, linear in N; representations are L2-normalized so routing, gate, and hash all act on the same angular quantity.

Under supervised routing, LSH nearly matches block-sparse and far exceeds centroid: MQAR accuracy 0.76 vs. 0.82 (block-sparse) vs. 0.17 (centroid) at n_{blk}=16, and 0.92 vs. 0.96 at n_{blk}=64, with recall plateauing at hit = 0.84 (an irreducible hash-collision miss). Linear selection *can* therefore preserve accuracy when the representation stays lossless, which the lossy centroid path cannot. This is the section's main positive result.

The open part is training without supervision. The routing loss above uses the known needle block, available only on synthetic MQAR. We tested the unsupervised regime two ways. On MQAR under gate coupling alone, LSH is fragile: recall is highly sensitive to the cosine gate temperature (hit 0.09, 0.34, 0.02 at scale 5.66, 30, 50) and never reaches block-sparse's gate-only level (acc 0.96, hit 0.78). On a real language model (TinyStories, 45M, gate coupling only) the result is sharper (Table 2): gate-only LSH reaches the *same* perplexity as own-block-only attention (no routing), and its perplexity does not improve with context length, whereas block-sparse beats the local baseline by 3.7 perplexity and improves with length. LSH attends to five blocks but adds nothing over attending to one: its content routing is inert.

Table 2: Gate-only TinyStories (45M, B_s=16, L=512, 3000 steps). Local-only is top-k=0 (own block, no routing). LSH matches the no-routing baseline; only block-sparse routing helps and improves with length.

	val PPL	PPL@128	PPL@256	PPL@512
Local-only (no routing)	21.43	20.7	20.6	21.7
LSH (linear routing)	21.49	21.0	21.0	21.7
Block-sparse routing	**17.69**	20.1	18.6	**17.8**

Across these runs one mechanism recurs: hard hashing gives the selector no gradient on blocks it does not already collide with, so when the selector is trained *through* the hash it cannot learn to route without supervision, while block-sparse's all-pairs top-k scores every block on every step and learns to route in every regime.

This points to what the inference-cost-only objective permits: train through the dense all-pairs top-k selector (full gradient, identical to block-sparse) and swap in LSH bucketing only at inference, on the same normalized representations, so the hash never enters the gradient path.

For *retrieval* this works and scales. On MQAR, LSH inference recovers the dense-trained selector at a fixed inference budget, and recall holds (even improves) as context grows: at 8 rounds and 64 buckets (constant, so inference is $\mathcal{O}(n_{\text{blk}})$), n_{blk}=128/256/512 give hit 0.978/0.992/0.994 (Table 3). The single needle is reliably hashed into the query's bucket.

For *language modeling* it does not. At n_{blk}=128 (block 4, L=512, gate only) and a matched attention budget (own block plus top-4), a dense selector that scores all blocks reaches 7.9 perplexity, but a parameter-free LSH selecting its top-4 from ∼16 bucket candidates reaches 113 to 1115 (worse at fewer rounds). The dense selector at the same budget succeeds, so this is not a coverage floor: the random hash fails to recall the handful of relevant blocks the dense selector finds. A single needle is easy to hash to; the blocks a language model needs per query are not, at least with a fixed random hash.

A *learned* hash removes the catastrophe. We make the projection R a trained parameter and add a self-distilled collision loss, an InfoNCE that pulls each query block's dense-selected key blocks to share a bucket under R, co-training the block representations to be hash-friendly with weight λ_h; the hash still never enters the inference selection's gradient path. The outcome is a tradeoff set by λ_h (Table 4). Light pressure (λ_h=0.1) leaves the dense oracle almost intact (6.6 to 7.8 perplexity) yet collapses the inference recall gap from 53× to 1.3×: the linear-inference LM reaches 10.2 perplexity, within 30% of the quadratic block-sparse selector at the same attention budget, where the random hash was unusable at 349. Heavier pressure (λ_h=0.5) closes the gap almost entirely (1.06×) but over-degrades the oracle (15.1), so net quality is worse. A learned hash thus buys recall, but only by reshaping the representation, which costs selection accuracy: the same representation-quality wall that defeats centroid routing, now seen from the hashing side.

Two controls show this coupling is intrinsic, not an artifact of co-training. Training only the hash planes while freezing the representations keeps the oracle intact (6.6) but leaves the inference gap 15× open (perplexity 97): the hash alone cannot recover recall. Annealing λ_h from 0.5 to 0 does not beat light constant pressure either (oracle 11.8, inference 14.0); once the representations are pulled toward hash-friendliness, relaxing the pressure only partly restores selection quality. Neither the hash nor the schedule separates recall from representation fidelity.

So train-dense / infer-LSH yields a genuinely linear selector that is accuracy-preserving for needle-style retrieval and, with a learned hash, usable for language modeling at a sparse budget, though still short of the quadratic block selector. This confirms at block granularity, and at small scale, the token-level finding of DASH-KV [20] that a learned hash recovers the recall a parameter-free hash [19] loses. Closing the remaining gap to the quadratic selector, rather than merely tracking a degraded oracle, is the open problem.

Table 3: Train dense, infer LSH at a *fixed* inference budget (8 rounds, 64 buckets, cap 8; $\mathcal{O}(n_{\text{blk}})$). Recall holds and improves as context grows. MQAR, supervised; oracle is the dense-trained selector.

n_{blk}	LSH acc	LSH hit	oracle acc
128	0.95	0.978	0.99
256	0.98	0.992	0.99
512	0.99	0.994	0.99

Table 4: Learned vs. random hash, train-dense / infer-LSH on a 45M language model (n_{blk}=128, block 4, top-4, 8 rounds, 64 buckets, gate only, 3000 steps). λ_h weights the collision loss; "R only" freezes the representations and trains only the hash planes; "anneal" decays λ_h from 0.5 to 0. Light constant pressure is the best operating point. The block-sparse matched-budget oracle (quadratic selection) is 7.9.

selector	oracle PPL	LSH-infer PPL	gap
random hash (λ_h=0)	6.6	348.6	53×
learned, R only (reps frozen)	6.6	97.4	15×
learned (λ_h=0.1)	7.8	**10.2**	1.3×
learned (λ_h=0.5)	15.1	16.1	1.06×
learned, anneal (λ_h: 0.5→0)	11.8	14.0	1.2×

6 Language Model Experiments

6.1 Setup

- **Dataset:** TinyStoriesV2-GPT4 [12], 5M tokens, GPT-2 BPE tokenizer (vocab 50,257).
- **Model:** 6 layers, $d = 512$, $h = 8$, weight-tied embeddings. 44.9M (dense) / 45.1M (sparse) parameters.
- **Training:** 5000 steps, batch 16, $L = 512$, AdamW (β_1=0.9, β_2=0.95, wd=0.1), cosine LR 3×10^{-4} with 200-step warmup. bf16 mixed precision.
- **Sparse config:** $B_s = 64$, top-$k = 4$, gate coupling, Triton kernel.
- **Hardware:** NVIDIA GB10 Grace-Blackwell, 128 GB unified memory.

6.2 Results

Table 5: TinyStories language model: dense vs. block-sparse Triton.

Model	Params	Val PPL	PPL@128	PPL@256	PPL@512	Time
Dense	44.9M	14.41	15.41	14.54	14.09	736 s
Sparse-Triton	45.1M	15.00	15.31	**14.16**	14.51	908 s

The sparse-Triton model achieves val perplexity 15.0 versus 14.4 for dense, a 4% gap. Notably, at 256-token context, sparse *beats* dense (14.16 vs. 14.54), suggesting the learned selector provides a useful inductive bias at moderate lengths. The training time overhead (908 s vs. 736 s, +23%) is from the selector computation and the Triton kernel's contiguity copies.

6.3 NIAH (Needle-in-a-Haystack)

Both models score 0% on NIAH at $L = 512$ (50 samples). This is a model capacity limitation (45M params, 3000 to 5000 steps), not a sparse attention defect. NIAH retrieval requires 100M+ parameters and 10k+ training steps.

7 Benchmark: Latency and Memory

Table 6: Forward-pass benchmark: D=512, H=8, B=1, B_s=128, k=8, causal. GB10 GPU.

L	Dense (ms)	Sparse-PT (ms)	Triton (ms)	Dense (GB)	PT (GB)	Triton (GB)
4,096	0.25	11.64	1.57	0.02	0.70	0.05
16,384	3.05	44.08	6.80	0.10	2.70	0.10
65,536	48.20	178.59	28.32	0.30	10.70	0.30
131,072	195.38	365.72	56.79	0.58	21.38	0.57
262,144	796.59	739.32	**114.37**	1.12	42.72	**1.11**

Memory. The Triton kernel uses 1.11 GB at 262k, at parity with dense FlashAttention (1.12 GB) and 40× less than the PyTorch gather reference (42.72 GB). The fused kernel never materializes the gathered K/V tensor; all intermediate state lives in SRAM.

Latency. The Triton kernel is 7.0× faster than dense at 262k (114 ms vs. 797 ms). The crossover lies between 32k and 65k. At 65k, the kernel is already 1.7× faster than dense.

PyTorch gather is the bottleneck. The same algorithm, same selection, but PyTorch materializes the gather (42.7 GB) and is 6.5× slower than Triton at 262k. The fused kernel proves the sparse mechanism itself is strictly superior to dense at long contexts; the reference implementation was the obstacle.

7.1 Selection vs. the Attention Read: the $N^{4/3}$ Floor

The benchmark above measures the fused attention *read*, which is linear. The block-pair *selection* (scoring all n_{blk}^2 block pairs, Section 3) is not. We isolate the two stages and sweep L to 16M tokens across block sizes (Table 7).

Table 7: Selection (quadratic) vs. attention read (linear), isolated. D=512, H=8, B_s=64, k=8, causal, GB10 (130 GB). "OOM": the $\mathcal{O}(n_{\text{blk}}^2)$ score matrix exceeds memory.

L	Selection (ms)	Read (ms)	Select mem (GB)
262,144	5.8	17.1	0.4
1,048,576	51.2	84.7	3.0
4,194,304	652.3	359.1	34.4
8,388,608	**OOM**	716.0	>130
16,777,216	**OOM**	1,447.6	>130

Scaling. Fitting across L, the attention read scales with exponent 0.96 to 1.02 (linear) at every block size, while selection scales with exponent 1.5 to 1.9 (quadratic, approaching 2 as N leaves the latency floor). Selection latency overtakes the read at ∼512k tokens for B_s=32 and ∼2M for B_s=64; for B_s=128 it stays below the read through 8M (ratio 0.17), crossing over only in the hundreds of millions.

The quadratic dies by OOM, not by latency. The $\mathcal{O}(n_{\text{blk}}^2)$ score matrix is a memory wall reached well before the latency crossover: it OOMs at 4M (B_s=32), 8M (B_s=64), and 16M (B_s=128), while the linear read runs to 16M (86 GB) on the same GPU. In practice, quadratic selection becomes impossible to hold, not merely slow.

Variable block size does not rescue linearity. A natural trick is to grow the block with context, $B_s \propto \sqrt{N}$, making selection $\mathcal{O}((N/\sqrt{N})^2) = \mathcal{O}(N)$, linear. We confirm this: along a $\sqrt{N}$ schedule (B_s=32, 64, 128 at L=256k, 1M, 4M) the measured selection exponent falls to 0.98. But the read processes kB_s keys per query, so it then grows as $NB_s \propto N^{1.5}$ (measured exponent rising to 2.3): the quadratic merely relocates from selection into the read. Writing the total as $N^2 s/B_s^2 + NcB_s$ and minimizing over B_s gives $B_s \propto N^{1/3}$, at which *both* terms equal $\mathcal{O}(N^{4/3})$. This $N^{4/3}$ is a floor: **no block-size schedule makes single-level, all-pairs block selection linear end-to-end; the quadratic can be moved between stages but not removed.** Genuinely linear selection requires scoring $\mathcal{O}(C)$ candidates per query (clustering, hashing, or hierarchy), not all n_{blk}, the open problem of Section 5.

8 Donor-Model Validation: Swap into Qwen3.6-35B-A3B

To test whether content-dependent block-sparse selection transfers beyond our 45M models, we swap it, at inference only, into a frontier open-weight model: Qwen3.6-35B-A3B, a 35B-parameter (3B active) mixture-of-experts model. The model is a hybrid: of its 40 layers only 10 are full softmax attention (every fourth; the other 30 are linear attention), so it is already mostly sub-quadratic. We replace the softmax attention in those 10 residual quadratic layers with our block-sparse select-then-attend, reusing the model's own query/key/value projections, partial multimodal RoPE, grouped-query attention (16 query heads, 2 key/value heads, head dimension 256), and output gate; only the dense softmax over all keys is replaced. Selection is training-free: each query block scores key blocks by a Quest-style [16] estimate of $\max_{t,s} q_t \cdot k_s$ (per-block key min and max), and attends to the top-k plus the own and sink blocks. The swap is a HuggingFace transformers

monkeypatch (not a vllm change); the block-sparse forward is the chunked PyTorch path of Section 3, so the whole study runs on a single GB10.

Correctness. With all blocks selected, the patched path reproduces the stock perplexity to 0.0000% at every context (bit-identical SDPA), confirming the integration; the chunked gather path matches it to 5.7×10^{-7} (fp32) and 4.6×10^{-3} (bf16) per output. Because the MoE router is discrete, a bf16-level attention perturbation in 10 layers amplifies into a small perplexity shift, so we baseline each sparse run against *all-blocks gather* (full attention through the same code path): the amplification is then common-mode and the delta isolates selection. That within-path floor is +0.28% at 16k.

Results. We measure held-out perplexity on a long English text (Project Gutenberg) from 16k to 64k context (Table 8). Training-free content selection recovers full-attention quality and degrades gracefully as the budget shrinks: at 64k, attending to 32 of 512 blocks (6%) stays within 1.60% of full attention. It beats a random-block baseline by 3 to 4× at matched budget on perplexity, and the gap *grows* with context (top-16: 4.1, 5.5, 7.5 percentage points at 16k, 32k, 64k), so selection matters more as context lengthens.

Table 8: Block-sparse swap into the 10 full-attention layers of Qwen3.6-35B-A3B, training-free, perplexity delta vs. stock (parentheses: fraction of blocks attended). Held-out Gutenberg text, B_s=128.

context	top-8	top-16	top-32	random-16
16k (n_{blk}=128)	+3.79% (6%)	+1.45% (13%)	+0.48% (25%)	+5.51%
32k (n_{blk}=256)	+5.43% (3%)	+2.33% (6%)	+0.97% (13%)	+7.86%
64k (n_{blk}=512)	+10.1% (2%)	+3.69% (3%)	+1.60% (6%)	+11.2%

How much headroom? The oracle ceiling. To bound what *any* top-k block selector could achieve, we replace the heuristic score with the stock model's own per-block attention mass, an oracle that selects the k highest-mass blocks (Table 9). The oracle roughly halves the heuristic's degradation, and the headroom grows with context: at top-16 the heuristic-to-oracle gap widens from 0.6 to 2.2 percentage points across 16k to 64k (at 64k, +3.69% falls to +1.45%), and at top-32 from 0.2 to 1.1 (at 64k, +1.60% to +0.52%). The training-free heuristic is thus strong but not optimal: real selection headroom exists, and it is largest in the long-context, moderate-budget regime where sparse attention matters most. At the most aggressive budget (top-8 at 64k) the oracle's advantage falls within measurement noise; only two non-overlapping windows fit at this length.

Table 9: Oracle ceiling. Perplexity delta vs. stock (%) for the training-free max heuristic and an oracle that selects the top-k blocks by the stock model's true per-block attention mass. The oracle lower-bounds the achievable delta for any top-k block selector; the heuristic-to-oracle gap is the recoverable headroom, which grows with context. Same evaluation windows as Table 8.

	top-8		top-16		top-32	
context	max	oracle	max	oracle	max	oracle
16k	+3.79	+1.75	+1.45	+0.83	+0.48	+0.24
32k	+5.43	+3.03	+2.33	+1.17	+0.97	+0.42
64k	+10.1	+10.7	+3.69	+1.45	+1.60	+0.52

A learned selector does not help. We also distilled a small per-layer selector (linear maps of block-pooled query and key, trained by KL to the stock per-block attention mass on a short calibration set) and swapped it in. It *underperforms* the training-free heuristic (at 16k, top-8 +5.5% vs. +4.0%; top-16 +2.4% vs. +1.2%): a linear map on pooled post-RoPE representations cannot beat a direct $\max q \cdot k$ estimate, echoing the representation tradeoff of Section 5. The oracle of Table 9 shows this is not for want of headroom (a perfect

selector recovers roughly half of the heuristic's loss). The bottleneck is not the pooled representation: giving the learned selector per-block key min and max, the extremes the max $q \cdot k$ estimate uses, leaves its fit to the teacher block-mass unchanged (identical KL) and still does not beat the heuristic (16k, top-8 +4.6%, top-16 +3.2%). What it imitates, the average per-block attention mass, is a softer signal than the peak interaction the heuristic estimates directly; recovering the oracle's headroom needs a better training target than mass distillation, not richer pooling, and remains open.

Downstream retrieval: needle in a haystack. Perplexity is a weak proxy for long-context capability, so we test single-needle retrieval directly: a magic number is placed at varying depths in long filler text and the model is queried for it, scored by greedy match on the answer span (six needles per cell). The dense model retrieves perfectly at every context (Table 10), so the harness is sound. With the block-mean selector, retrieval holds only above a budget near 13%, well above the 6% at which perplexity stays within 1.6% (Table 10): at 32k, top-32 (13%) retrieves all six needles while top-16 (6%) retrieves none, and at 64k every tested budget ($\leq 6\%$) fails; where it works it beats random (0.67 vs. 0.0 at 16k top-16). So this selector needs about twice the perplexity budget to retrieve. Six needles per cell is coarse, so the budget floor and the dense ceiling are the robust signals, not the per-cell rates. Most of this floor is the selector's query pooling, not content. SelectMax scores key blocks against the block-mean query, which ranks the needle block near 13% of all blocks and worsens with context (median rank 19, 28, 67 at 16k, 32k, 64k), whereas the lone retrieval query token ranks it near 8 at every length: pooling B_s queries dilutes the one that retrieves. Selecting by each query block's last token instead (a one-line change, and exactly the query an autoregressive decode step issues) largely closes the floor: it lifts 16k top-16 from 0.67 to 1.0 and, where the block-mean retrieves nothing, recovers 0.67 at 32k top-16 and at both top-16 and top-32 of 64k, tracking the single-token rank rather than a propagation limit. A two-stage mean-then-max variant helps less, since the per-token max re-inflates competitor blocks. The prefill block-mean, not content selection, thus sets most of the retrieval floor, and single-token decode should largely avoid it.

Table 10: Single-needle retrieval (exact match, six needles per cell) for the block-sparse swap. Each budget cell shows the block-mean selector / the last-token selector; the dense model retrieves perfectly (1.00) at every context. Block fractions for top-8/16/32 are 6/13/25% at 16k, 3/6/13% at 32k, 2/3/6% at 64k; random-16 is the block-mean selector over random blocks. The block-mean selector needs about 13% of blocks, while the last-token selector retrieves at the perplexity budget (64k top-32, 6%) and below.

context	top-8 mean / last	top-16 mean / last	top-32 mean / last	random-16 mean
16k	0.00/0.50	0.67/1.00	0.67/1.00	0.00
32k	0.00/0.00	0.00/0.67	1.00/1.00	0.33
64k	0.00/0.33	0.00/0.67	0.00/0.67	0.00

Scope. This is an inference-time, forward-only perplexity study on in-distribution text; the relative deltas, not absolute perplexity, are the signal. It measures perplexity and single-needle retrieval (Table 10); broader suites (RULER, multi-needle) and training with the swap remain future work, as is a grouped-query, RoPE-aware fused kernel for wall-clock gains at these contexts.

9 Discussion

9.1 What Works

Gate coupling + auxiliary routing loss reliably trains the block selector (hit 0.99 on MQAR, val PPL within 4% of dense on TinyStories [12]). The fused Triton kernel delivers the theoretical memory and latency advantages of block-sparse attention in practice, with no accuracy loss from numerical approximation.

9.2 What Doesn't: Centroid Routing

Centroid routing achieves linear selection cost and perfect retrieval recall (hit=1.0) but cannot match block-sparse task accuracy (acc 0.16 vs. 0.77). The failure is not in routing (InfoNCE [14] fixes that) but in *representation quality*: the coarse centroid path shapes the attention projections differently, limiting fine-grained token extraction. This is a fundamental limitation of content-routed linear-selection attention that, to our knowledge, has not been previously characterized.

9.3 Linear Selection and What Scales Instead

The LSH study (Section 5.6) sharpens the centroid finding. A learned hash can close the train-dense / infer-LSH recall gap that a parameter-free hash leaves open (from 53× to 1.3× on the 45M LM), but only by co-training the block representations to be hash-friendly, which degrades the selection oracle. Recall and representation fidelity therefore trade off: centroid routing sacrifices the representation outright, a learned hash sacrifices a tunable fraction of it, and neither reaches the quadratic block selector at a sparse budget. Two controls (Section 5.6) show the coupling is intrinsic: freezing the representation and learning only the hash leaves the gap 15× open, and annealing the hash weight to zero does not beat light constant pressure. This is consistent with what scales in practice. Trained block-selection methods (NSA [3], MoBA [4]) do not make selection asymptotically linear: they score blocks with a trained selector and attend to a selected linear-size subset, paying an $\mathcal{O}(n_{\text{blk}}^2)$ block-scoring cost that is sub-quadratic in N only by the block-size constant. The $N^{4/3}$ floor (Section 7.1) explains why this is the design that scales: no block schedule removes the quadratic from a single-level all-pairs selector, so the practical compromise is to keep selection quadratic but cheap and trainable and to save the large constant on the attention read, exactly the regime the fused kernel targets. Genuinely linear, accuracy-preserving content selection remains open.

9.4 Limitations

- Single-level block-pair selection is $\mathcal{O}(n_{\text{blk}}^2)$ with an $\mathcal{O}(N^{4/3})$ floor no block schedule can beat (Section 7.1); it exhausts memory before any latency crossover. The linear selectors trade cost for quality: centroid routing is accuracy-broken, and a learned-hash LSH is usable but representation-taxed, still short of the quadratic selector.
- NIAH on our from-scratch models needs more scale (the 45M scores zero); we measure retrieval instead on the 35B donor swap (Section 8), where the usable budget depends strongly on the query reduction.
- The backward kernel uses `atomic_add` for dK/dV, which may serialize under high contention. A split-K or warp-level reduction could improve throughput.
- No autoregressive KV-cache support; the kernel is training/prefill only.

10 Conclusion

We demonstrated that block-sparse attention with a fused Triton kernel achieves memory parity with dense FlashAttention and 7.0× latency speedup at long contexts, while maintaining within-4% perplexity on a real language model. The gate-coupled selector training recipe is simple, effective, and requires no separate pre-training stage.

We probed two linear-time selectors. Centroid routing reaches linear cost and perfect recall but degraded accuracy: the obstacle is representation quality under coarse routing, not routing recall (InfoNCE resolves the latter). LSH bucketing, which prunes the search rather than the representation, matches block-sparse accuracy under supervised routing and far exceeds centroid, showing a lossless-refine linear selector can preserve accuracy; but its unsupervised routing proved inert, matching a no-routing baseline on a 45M language model. Reliable unsupervised linear selection remains open. Training through a dense selector and swapping in a learned hash at inference makes linear-inference language modeling usable, within 30% of the quadratic selector at a sparse budget, though closing that gap remains open.

Beyond our own models, the mechanism transfers: swapping the training-free selector into the 10 full-attention layers of the 35B-parameter Qwen3.6-35B-A3B holds within 1.6% of full-attention perplexity at

64k context while attending to 6% of blocks, evidence that content-dependent block selection is useful in a frontier model at inference, with no training.

Promising directions include scaling our from-scratch models to the 100M+ regime where needle-in-a-haystack retrieval becomes measurable on them (it already is on the 35B donor swap, Section 8), extending the kernel to autoregressive KV-cache decoding, and designing a linear-selection mechanism that preserves the representation quality of pairwise scoring. More broadly, the fused kernel removes the implementation barrier that has masked block-sparse attention's asymptotic advantage: at long contexts, content-dependent sparsity is now both faster and lighter than dense attention in practice, not merely in theory.

References

[1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.14135.

[2] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.08691.

[3] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y.-X. Wei, L. Wang, Z. Xiao, Y. Wang, C. Ruan, M. Zhang, W. Liang, and W. Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint* arXiv:2502.11089, 2025. ACL 2025 Best Paper.

[4] E. Lu, Y. Duan, H. Hu, W. Wang, J. Zhu, H. Liu, K. Sun, and T. Wang. MoBA: Mixture of block attention for long-context LLMs. In *NeurIPS*, 2025. arXiv:2502.13189.

[5] A. Roy, M. Saffar, A. Vaswani, and D. Grangier. Efficient content-based sparse attention with routing transforms. *Transactions of the Association for Computational Linguistics (TACL)*, 9:53-68, 2021.

[6] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V. Le, G. E. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1701.06538.

[7] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv preprint* arXiv:2004.05150, 2020.

[8] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontañón, et al. BigBird: Transformers for longer sequences. In *NeurIPS*, 2020. arXiv:2007.14062.

[9] N. Kitaev, Ł. Kaiser, and A. Levskaya. Reformer: The efficient transformer. In *ICLR*, 2020. arXiv:2001.04451.

[10] Y. Tay, D. Bahri, D. Metzler, D.-C. Juan, Z. Zhao, and C. Zheng. Sparse sinkhorn attention. In *ICML*, 2020. arXiv:2002.11296.

[11] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, B. Mann, et al. In-context learning and induction heads. *Transformer Circuits Thread*, Anthropic, 2022.

[12] R. Eldan and Y. Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint* arXiv:2305.07759, 2023.

[13] P. Tillet, H.-T. Kung, and D. D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proc. 3rd ACM SIGPLAN Int. Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10-19, 2019.

[14] A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint* arXiv:1807.03748, 2018.

[15] S. Arora, S. Gopi, N. Ingster, and P. Nakkiran. Simple, fast, and practically good: A simple LLM-based sparse retrieval model for MQAR. 2024.

[16] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *ICML*, 2024. arXiv:2406.10774.

[17] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023. arXiv:2306.14048.

[18] D. Liu, M. Chen, B. Lu, H. Jiang, Z. Han, Q. Zhang, Q. Chen, C. Zhang, B. Ding, K. Zhang, C. Chen, F. Yang, Y. Yang, and L. Qiu. RetrievalAttention: Accelerating long-context LLM inference via vector retrieval. *arXiv preprint* arXiv:2409.10516, 2024.

[19] T. Rabbani, M. Liu, T. O'Halloran, A. Sankaralingam, M.-A. Hartley, and F. Huang. LSH-E tells you what to discard: An adaptive locality-sensitive strategy for KV cache compression. In *NeurIPS Workshop on Machine Learning and Compression*, 2024.

[20] J. Guo, Z. Zhang, J. Xie, M. T. Iqbal, D. Han, L.-H. Lee, S.-H. Bae, J. Zou, Y. Yang, and C. Zhang. DASH-KV: Accelerating long-context LLM inference via asymmetric KV cache hashing. *arXiv preprint* arXiv:2604.19351, 2026.